# TranCI

Configuration-interaction calculations for a transition-metal $d$ shell

User manual and physical reference

**Abstract**

TranCI solves the many-body problem of a single transition-metal $d$ shell exactly, by full configuration interaction (exact diagonalization) in the space of $n_e$ electrons distributed over the ten $d$ spin-orbitals. The Hamiltonian combines the full Coulomb repulsion between the $d$ electrons, crystal fields of arbitrary symmetry, spin–orbit coupling, and magnetic and exchange fields. From the resulting many-body spectrum the code extracts level degeneracies, excitation energies, expectation values of spin and orbital operators, orbital occupations, correlation entropies, dynamical correlation functions, and low-energy effective spin Hamiltonians. This document describes the physics that the code implements, the exact form of every operator it provides, and how to drive it both from Python and from the graphical interface.

## Contents

# 1 Overview

## 1.1 What the code solves

A transition-metal ion in a solid, on a surface, or in a molecule carries an open $d$ shell. Because the $d$ orbitals are compact, the Coulomb repulsion between the $d$ electrons is comparable to, or larger than, the crystal-field splitting induced by the environment. A single-particle (mean-field) picture is then not sufficient: the multiplet structure — Hund's rules, the ordering of the $^3F$, $^3P$, $^1G$ ... terms, magnetic anisotropy generated by the interplay of spin–orbit coupling and the crystal field — is a genuinely many-body effect.

TranCI solves the problem without approximation inside the $d$ shell. It constructs the full many-body Hamiltonian in the basis of all Slater determinants with $n_e$ electrons in ten spin-orbitals, and diagonalizes it exactly. Because the largest space has only $\binom{10}{5} = 252$ states, this is cheap, and every eigenstate is available.

## 1.2 Structure of a calculation

Every calculation follows the same three steps.

1. **Load the operator library for a given occupation.** `get_atom(ne)` returns an `Atom` object holding a dictionary `Atom.Operator` of *many-body* matrices — all of them already expressed in the $\binom{10}{n_e}$-dimensional basis of that electron number.
2. **Assemble the Hamiltonian yourself** as a linear combination of those matrices. There is no fixed model: the Hamiltonian is whatever Hermitian combination you write down.
3. **Diagonalize and analyse**, via `Atom.get_manifolds(H)`, which returns a `Lowest_States` object exposing the spectrum, the degenerate manifolds and all the observables.

```
import sys; sys.path.append("<repo>/src")
from tranci.atom import get_atom

Atom = get_atom(ne=3) # d^3 ion, 120 many-body states
V = Atom.Operator["Coulomb"] # electron-electron repulsion
CF = Atom.Operator["z2"] # uniaxial crystal field
LS = Atom.Operator["ls"] # spin-orbit coupling
Sz = Atom.Operator["sz"] # total S_z

H = 4*V + 0.3*CF + 0.05*LS + 0.01*Sz # your Hamiltonian, in eV
M = Atom.get_manifolds(H) # diagonalize

print(M.get_gs_degeneracy()) # (degeneracy, energy)
print(M.get_excitations()) # excitation energies [eV]
print(M.get_gs_projected_eigenvalues(Sz))
```

**Units.** All energies are in electronvolts (eV) everywhere in the library. The only exception is the file `SPECTRUM.OUT` written by the graphical interface, which additionally reports energies in GHz and meV.

## 1.3 Installation and imports

The library is *not* installed as a Python package. Every script must prepend the source directory to the module search path:

```
import sys
sys.path.append("/path/to/tranci/src")
from tranci.atom import get_atom
```

The bundled examples locate the repository relative to the *current working directory*, not to the script file, so they must be run from inside their own directory:

```
cd examples/octahedral && python main.py
```

Running `python examples/octahedral/main.py` from the repository root fails on import. The notebooks in `notebooks/` resolve the repository the same way, so their kernel must be started with the working directory in `notebooks/`.

Dependencies are `numpy`, `scipy` and `matplotlib` for the core library; `PyQt5` and a `pdflatex` installation for the graphical interface; and `jax` for the effective-Hamiltonian fitting. `numba` is used if present but is optional.

# 2 The model

## 2.1 Single-particle space

The $d$ shell has $\ell = 2$, hence $2\ell + 1 = 5$ orbital states labelled by the magnetic quantum number $m = -2, -1, 0, +1, +2$, each of which can hold a spin-up or a spin-down electron. The single-particle space therefore has ten spin-orbitals $|m, \sigma\rangle$. The code orders them with spin as the slow index and $m$ as the fast index:

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $m$ | $-2$ | $-1$ | $0$ | $+1$ | $+2$ | $-2$ | $-1$ | $0$ | $+1$ | $+2$ |
| $\sigma$ | $\uparrow$ | $\uparrow$ | $\uparrow$ | $\uparrow$ | $\uparrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ | $\downarrow$ |

This ordering is recorded in the file `orbital.in` shipped with each operator library, and it is the ordering in which every $10 \times 10$ single-particle matrix must be written when it is supplied by the user (Sec. **??**).

## 2.2 Many-body space

For $n_e$ electrons the many-body basis consists of all occupation-number states (Slater determinants)

$$|n_0 n_1 \dots n_9\rangle \;=\; \prod_{i:\, n_i = 1} c_i^{\dagger}\, |0\rangle, \qquad n_i \in \{0, 1\}, \qquad \sum_i n_i = n_e, \tag{1}$$

with the creation operators applied in increasing index order, so that fermionic signs are fixed unambiguously. The dimension is the binomial coefficient $\binom{10}{n_e}$:

| $n_e$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| $\binom{10}{n_e}$ | 10 | 45 | 120 | 210 | 252 | 210 | 120 | 45 | 10 |

The occupation vectors, one per row, are stored in `basis.out` inside each operator library, and are read into `Atom.basis` as a list of `BaseMB` objects. Occupations $n_e = 1$ through 9 are supported; $n_e = 0$ and $n_e = 10$ are trivial closed-shell cases and are rejected.

## 2.3 Where the matrices come from

The many-body matrices are not built at run time. They are precomputed once by a C++ generator (`src/tranci/cpplib/`) and stored, in sparse text form, in `src/tranci/cilib/<ne>/`. Each `.op` file starts with a header line `# SIZE = d` and then lists `i j real imag` for the non-zero elements of a Hermitian matrix. `get_atom(ne)` simply reads that directory. This is why loading an atom is instantaneous and why the physical content of the Coulomb tensor (Sec. **??**) is fixed by the shipped data.

# 3 The Hamiltonian

The Hamiltonian is written by the user as a linear combination of the operators in `Atom.Operator`. A typical complete Hamiltonian reads

$$\mathcal{H} \;=\; U\,\hat{V}_{ee} \;+\; \hat{V}_{\mathrm{CF}} \;+\; \lambda \sum_i \vec{l}_i \cdot \vec{s}_i \;+\; \mu_B \vec{B} \cdot (\vec{L} + 2\vec{S}) \;+\; \vec{J} \cdot \vec{S}, \tag{2}$$

All coefficients are in eV, with the single exception of the Coulomb prefactor $U$, which is a dimensionless multiplier (Sec. **??**). The rest of this section gives the precise operator that each dictionary key corresponds to.

## 3.1 Electron–electron interaction

The key `"Coulomb"` (equivalently `"vc"`) holds the full two-body interaction inside the $d$ shell,

$$\hat{V}_{ee} \;=\; \sum_{ijkl} \sum_{\sigma\sigma'} V_{ijkl}\; c^\dagger_{i\sigma} c^\dagger_{j\sigma'} c_{k\sigma'} c_{l\sigma}, \tag{3}$$

with $i, j, k, l$ running over the five orbital indices and the spin sums running over the two spin channels. The interaction is spin-independent and rotationally invariant, so it conserves $m_i + m_j = m_k + m_l$: only the 85 symmetry-allowed components are non-zero. Note that Eq. (**??**) carries no factor $\frac{1}{2}$; the tensor $V_{ijkl}$ read from file is correspondingly one half of the conventional Coulomb matrix element.

The tensor is the standard Slater–Condon decomposition of the Coulomb kernel for $\ell = 2$,

$$V_{ijkl} \;\propto\; \sum_{\kappa=0,2,4} F^\kappa \; c^\kappa(m_i, m_l)\, c^\kappa(m_j, m_k), \tag{4}$$

where the $c^\kappa$ are the usual Gaunt coefficients for $\ell = 2$ and the sum runs over the even ranks $\kappa = 0, 2, 4$ allowed by parity. All 85 tabulated magnitudes are reproduced exactly by a single set of Slater integrals. (The sign convention of the stored table is tied to the $c^\dagger c^\dagger c\, c$ operator ordering used by the generator and is not a separate physical input.)

The physically meaningful statement is the one about the operator that the user actually applies. With a multiplier $u$, i.e. for $u \cdot$ `Atom.Operator["Coulomb"]`, the effective Slater and Racah parameters are

$$F^0 = 7.70\,u \text{ eV}, \qquad F^2 = 4.06\,u \text{ eV}, \qquad F^4 = 2.65\,u \text{ eV}, \tag{5}$$

$$B = \frac{F^2}{49} - \frac{5F^4}{441} = 52.9\,u \text{ meV}, \qquad C = \frac{35F^4}{441} = 210.3\,u \text{ meV}, \qquad C/B = 3.98, \tag{6}$$

in line with the textbook value $C/B \approx 4$ for $3d$ ions and with the atomic ratio $F^2/F^4 \simeq 1.53$. These values are not inferred from the file but measured directly from the computed $d^2$ multiplet spectrum, which they reproduce to all printed digits (Sec. **??**).

**Important: the prefactor is a multiplier, not $F^0$.** The number you place in front of `Atom.Operator["Coulomb"]` multiplies this *whole fixed tensor*; it is dimensionless, not an energy. Writing `H = 4*V` therefore means

$$F^0 = 30.8 \text{ eV}, \quad F^2 = 16.2 \text{ eV}, \quad F^4 = 10.6 \text{ eV}, \quad B = 212 \text{ meV}, \quad C = 841 \text{ meV}, \tag{7}$$

which is a very strongly correlated regime: the $d^2$ term splittings are then of order 3 eV. The same holds for the field labelled `U [multiplier]` in the graphical interface: its default value $U = 4$ scales the tensor by 4. Treat $U$ as a dimensionless strength of the multiplet interaction and tune it so that the term splittings match the system you model. Free $3d$ ions have $B \approx 0.10$–$0.13$ eV, reduced by 20–40% in a solid, which corresponds to $u \approx 1.5$–$2.5$.

Note finally that $F^0$ only shifts the total energy at fixed $n_e$, so it never affects a spectrum computed at fixed occupation — only $B$ and $C$ matter.

### 3.2 Crystal fields

The environment of the ion — ligands, a surface, a molecular cage — lowers the spherical symmetry of the free ion and splits the five $d$ orbitals. TranCI supplies a set of Cartesian crystal-field generators built from powers of the single-particle orbital angular momentum:

$$\texttt{x2} = \sum_i (l_x^2)_i, \quad \texttt{y2} = \sum_i (l_y^2)_i, \quad \texttt{z2} = \sum_i (l_z^2)_i, \quad \texttt{x4} = \sum_i (l_x^4)_i, \quad \text{etc.} \tag{8}$$

$$\texttt{x2y2} = \sum_i \Big[ (l_x l_y)^2 + (l_y l_x)^2 \Big]_i. \tag{9}$$

> **These are sums of single-particle operators, not powers of the total angular momentum.** That is, $\texttt{x2} = \sum_i (l_x^2)_i \neq L_x^2$. The many-body squares $L^2$, $S^2$, $J^2$ are available separately as `"l2"`, `"s2"`, `"j2"` and are built as $L_x^2 + L_y^2 + L_z^2$ from the total operators. Confusing the two is the most common source of unexpected spectra.

**Uniaxial field (`z2`).** $D \sum_i (l_z^2)_i$ has single-particle eigenvalues $m^2 D$, so it splits the $d$ shell as

$$E(d_{z^2}) = 0, \qquad E(d_{xz}), E(d_{yz}) = D, \qquad E(d_{xy}), E(d_{x^2-y^2}) = 4D. \tag{10}$$

For $D > 0$ the $d_{z^2}$ orbital is stabilized. This is the tetragonal / axially-compressed geometry; $D < 0$ reverses the ordering.

**Rhombic field (`x2−y2`).** $E \sum_i (l_x^2 - l_y^2)_i$ breaks the remaining $xy$ degeneracy. Its single-particle spectrum is $\{\pm 2\sqrt{3},\, \pm 3,\, 0\} \times E$.

**Octahedral field (`x4 + y4 + z4`).** The cubic invariant $O \sum_i (l_x^4 + l_y^4 + l_z^4)_i$ has exactly two distinct single-particle eigenvalues: $18\,O$ (six-fold: the $t_{2g}$ triplet $d_{xy}, d_{xz}, d_{yz}$, times spin) and $24\,O$ (four-fold: the $e_g$ doublet $d_{z^2}, d_{x^2-y^2}$, times spin). For $O > 0$ the $t_{2g}$ states lie lowest, as in an octahedral ligand cage, and the ligand-field splitting is

$$10\,Dq \;=\; 6\,O. \tag{11}$$

For $O < 0$ the ordering is inverted, corresponding to a tetrahedral or cubic environment.

**Trigonal field.** The trigonal generator is the uniaxial field rotated so that its symmetry axis points along the body diagonal $\hat{n} = (1,1,1)/\sqrt{3}$,

$$\hat{V}_{\text{tri}} = t\, R \Big[ \textstyle\sum_i (l_z^2)_i \Big] R^\dagger = t \sum_i (\vec{l}_i \cdot \hat{n})^2 = \tfrac{t}{3} \sum_i \big( l_x + l_y + l_z \big)_i^2, \qquad R = e^{\,i\phi L_z} e^{\,i\theta L_x}, \tag{12}$$

with $\cos\theta = 1/\sqrt{3}$ and $\phi = \pi/4$, as implemented in `hamiltonians.build_hamiltonian`. The identity with $(\vec{l}\cdot\hat{n})^2$ has been verified numerically for the $(1,1,1)$ diagonal specifically. Note the factor $1/3$, which the interface label "$((x+y+z)/\sqrt{3})^2$" reflects: the operator is normalized so that its single-particle eigenvalues are again $0, 1, 4$, exactly as for `z2`. In your own scripts you can build any rotated field the same way, using the total $L_\alpha$ matrices as generators.

**Arbitrary crystal fields.** Any one-body crystal field can be supplied directly as a $10 \times 10$ matrix and promoted to the many-body basis; see Sec. **??**.

### 3.3 Spin–orbit coupling

The key `"ls"` holds the one-body spin–orbit operator summed over electrons,

$$\hat{H}_{\mathrm{SOC}} = \lambda \sum_i \vec{l}_i \cdot \vec{s}_i = \lambda \sum_i \Big[ (l_z s_z)_i + \tfrac{1}{2}(l_+ s_- + l_- s_+)_i \Big]. \tag{13}$$

For a single electron in the $d$ shell this operator has eigenvalues $-\frac{3}{2}$ (four-fold, $j = \frac{3}{2}$, verified through $\langle j^2 \rangle = \frac{15}{4}$) and $+1$ (six-fold, $j = \frac{5}{2}$), so a positive $\lambda$ places the $j = 3/2$ multiplet lowest and the splitting is

$$\Delta E_{5/2-3/2} = \tfrac{5}{2}\lambda. \tag{14}$$

**Use $\lambda > 0$ for every filling.** The input $\lambda$ is the *one-electron* spin–orbit constant, which is positive for all $d$ ions. The familiar sign change of the *effective* many-body coupling $\lambda_{LS} = \pm\zeta/2S$ above half filling is not something you put in: it emerges from the many-body problem, and the code reproduces it automatically. Diagonalizing $u\,\hat{V}_{ee} + \lambda\,\mathtt{ls}$ with $\lambda > 0$ gives, for the free-ion ground terms,

| $n_e$ | 2 | 3 | 7 | 8 |
|---|---|---|---|---|
| term | $^3F$ | $^4F$ | $^4F$ | $^3F$ |
| $J$ (computed) | 2 | 3/2 | 9/2 | 4 |
| Hund III | $\lvert L{-}S\rvert$ | $\lvert L{-}S\rvert$ | $L{+}S$ | $L{+}S$ |

i.e. $J = |L - S|$ below half filling and $J = L + S$ above it, exactly as Hund's third rule requires, with the same positive $\lambda$ throughout. Typical magnitudes are $\lambda \sim$ 10–100 meV for $3d$ ions, growing rapidly down the periodic table.

### 3.4 Magnetic and exchange fields

Two distinct field terms are available.

**Zeeman field.** Couples to the total magnetic moment,

$$\hat{H}_Z = \vec{b} \cdot (\vec{L} + 2\vec{S}), \tag{15}$$

using the many-body $L_\alpha$ and $S_\alpha$ (keys `"lx"`…, `"sx"`…). The vector $\vec{b}$ is entered in eV, i.e. it already includes the Bohr magneton: $b = \mu_B B$ with $\mu_B = 5.7884 \times 10^{-5}$ eV/T, so 1 T corresponds to $b \simeq 5.79 \times 10^{-5}$ eV. The factor 2 is the free-electron spin $g$-factor.

**Exchange field.** Couples to the spin only,

$$\hat{H}_J = \vec{J} \cdot \vec{S}, \tag{16}$$

which models the molecular field of a magnetically ordered substrate or of neighbouring magnetic ions. Because it does not act on the orbital sector, it is the right tool for isolating spin physics from orbital physics.

### 3.5 Hyperfine coupling

A spin-$\frac{1}{2}$ nucleus can be added with `hyperfine.add_nucleus(atom)`, which doubles the Hilbert space to $|\text{electronic}\rangle \otimes |I_z\rangle$ and provides $\vec{I}$ operators together with $\vec{S} \cdot \vec{I}$. The Hamiltonian then acquires

$$\hat{H}_{\mathrm{hf}} = A\,\vec{S} \cdot \vec{I} + g_n\,\vec{b} \cdot \vec{I}, \qquad g_n \simeq 1/1836, \tag{17}$$

with the nuclear Zeeman term scaled by the proton-to-electron mass ratio. This path is exercised only through `build_hamiltonian` when `atom.has_nucleus` is set; it is not exposed in the graphical interface.

# 4 Diagonalization, manifolds and degeneracies

## 4.1 Exact diagonalization

`Atom.get_manifolds(H)` converts the sparse Hamiltonian to a dense matrix and diagonalizes it completely with a Hermitian LAPACK routine. All $\binom{10}{n_e}$ eigenvalues and eigenvectors are computed and retained; no truncation of the many-body space is made anywhere. The cost scales as $\dim^3$, which for the worst case $\dim = 252$ is milliseconds.

The result is a `Lowest_States` object with

- `.evals` — the exact eigenvalues, *shifted so that the ground state is at zero*;
- `.e0` — the absolute ground-state energy before that shift;
- `.evecs` — the eigenvectors, one per row;
- `.h` — the Hamiltonian that was diagonalized;
- `.atom` — the originating `Atom` object.

## 4.2 Degenerate manifolds

Symmetry makes the spectrum highly degenerate, and almost every physical statement (degeneracy of the ground state, magnitude of a zero-field splitting, which excitations are allowed) requires grouping eigenvalues into manifolds. Two eigenvalues are declared degenerate when

$$|E_a - E_b| < \texttt{tol}, \qquad \texttt{tol} = 10^{-\texttt{ntol}}, \tag{18}$$

where `tol` and `ntol` are *module-level globals* of `tranci.hamiltonians` (default `ntol` = 6, i.e. `tol` $= 10^{-6}$ eV). The graphical interface overwrites them at run time from the "Energy tolerancy" field. From a script you set them explicitly:

```
from tranci import hamiltonians
hamiltonians.tol = 1e-5 # grouping window in eV
hamiltonians.ntol = 5
```

This window is a *grouping* parameter, not a display precision: `.evals` always holds the exact eigenvalues. Choosing it too large merges physically distinct levels (a small zero-field splitting disappears); choosing it too small splits a symmetry-degenerate manifold into numerically distinct pieces. A sensible value is well below the smallest physical splitting you care about, and well above the numerical noise of the diagonalization ($\sim 10^{-12}$ eV).

## 4.3 Ground-state degeneracy and excitations

- `get_gs_degeneracy(tol=None)` returns the **tuple** $(n_{\mathrm{GS}}, E_{\mathrm{GS}})$, where $n_{\mathrm{GS}}$ is the integer number of states within `tol` of the minimum. Take element `[0]` for the degeneracy; `get_gs_multiplicity()` returns that integer directly.
- `get_degeneracies()` returns the list $[(n_0, E_0), (n_1, E_1), \ldots]$ for all manifolds.
- `get_excitations()` returns $[E_k - E_0]$, the excitation energies of each manifold measured from the ground state, in eV.
- `get_manifolds()` returns the eigenvectors grouped by manifold; `get_gs_manifold()` returns only the ground-state group.

### 4.4 Disentangling a degenerate manifold

Inside a degenerate manifold the eigenvectors returned by the diagonalizer are an arbitrary basis. To get physically meaningful states one re-diagonalizes a chosen operator $A$ within the manifold:

$$A^{(\mathrm{mf})}_{ab} = \langle\psi_a|A|\psi_b\rangle, \qquad \text{diagonalize } A^{(\mathrm{mf})} \;\Rightarrow\; \text{new basis.} \tag{19}$$

`disentangle_manifolds(A)` does this for every manifold and rewrites `.evecs`; `disentangle_gs_manifold(A)` does it for the ground state only. Using $A = J_z$ (as the graphical interface does) labels the states by their total angular-momentum projection, which is what makes the printed wavefunctions readable.

## 5 Observables

### 5.1 Projected expectation values

The central tool is the representation of an operator inside a set of states,

$$A^{(\mathrm{mf})}_{ab} = \langle\psi_a|A|\psi_b\rangle, \tag{20}$$

available as `get_representation(A, n=6)` for the $n$ lowest states. For the ground-state manifold,

$$\texttt{get\_gs\_projected\_eigenvalues}(A) = \mathrm{eig}\big[A^{(\mathrm{GS})}\big] \tag{21}$$

returns the eigenvalues of $A$ restricted to the ground-state manifold. This is the quantity to look at for a degenerate ground state: for a spin doublet projected on $S_z$ it returns $\{-\frac{1}{2}, +\frac{1}{2}\}$; for an orbital projector it returns the occupations of that orbital in the manifold's natural basis, whose mean is the average occupation.

### 5.2 Angular momentum operators

`Atom.Operator` contains the many-body components

$$S_\alpha = \sum_i (s_\alpha)_i, \qquad L_\alpha = \sum_i (l_\alpha)_i, \qquad J_\alpha = L_\alpha + S_\alpha, \tag{22}$$

under the keys `"sx"`, `"sy"`, `"sz"`, `"lx"`…, `"jx"`…, together with the squares

$$S^2 = S_x^2 + S_y^2 + S_z^2, \qquad L^2 = L_x^2 + L_y^2 + L_z^2, \qquad J^2 = J_x^2 + J_y^2 + J_z^2, \tag{23}$$

under `"s2"`, `"l2"`, `"j2"`. From $\langle S^2\rangle = S(S+1)$ and $\langle L^2\rangle = L(L+1)$ one reads off the term symbol of any manifold:

$$S = \tfrac{1}{2}\left(-1 + \sqrt{1 + 4\langle S^2\rangle}\right), \qquad L = \tfrac{1}{2}\left(-1 + \sqrt{1 + 4\langle L^2\rangle}\right). \tag{24}$$

The commutation relations $[L_x, L_y] = iL_z$ etc., $[\vec{L}, \vec{S}] = 0$ and $[\vec{L}, \hat{V}_{ee}] = [\vec{S}, \hat{V}_{ee}] = 0$ are verified at run time by `tranci.check.check_all(atom)`.

### 5.3 Orbital projectors

Occupations of the cubic ($t_{2g}/e_g$) orbitals are obtained from projectors built out of the standard cubic harmonics,

$$|d_{z^2}\rangle = |m{=}0\rangle, \qquad |d_{x^2-y^2}\rangle = \tfrac{1}{\sqrt{2}}\big(|{+}2\rangle + |{-}2\rangle\big), \qquad |d_{xy}\rangle = \tfrac{i}{\sqrt{2}}\big(|{-}2\rangle - |{+}2\rangle\big), \tag{25}$$

$$|d_{xz}\rangle = \tfrac{1}{\sqrt{2}}\big(|{-}1\rangle - |{+}1\rangle\big), \qquad |d_{yz}\rangle = \tfrac{i}{\sqrt{2}}\big(|{-}1\rangle + |{+}1\rangle\big). \tag{26}$$

The corresponding many-body number operators

$$\hat{n}_\mu = \sum_\sigma c^\dagger_{\mu\sigma} c_{\mu\sigma} \tag{27}$$

are stored under the keys `"dz2"`, `"dx2y2"`, `"dxy"`, `"dxz"`, `"dyz"`. Each has trace 2 in the single-particle space (two spin channels), and together they resolve the identity. Additional projectors on the magnitude of the magnetic quantum number are available as `"m0"`, `"m1"`, `"m2"` (projecting onto $|m| = 0, 1, 2$), and the ten individual spin-orbital projectors are attributes `atom.um2`…`atom.dp2`.

Taking the mean of `get_gs_projected_eigenvalues(Atom.Operator["dxy"])` gives the average $d_{xy}$ occupation of the ground-state manifold; summing over the five orbitals returns $n_e$.

## 5.4 The $g$-tensor of a Kramers doublet

When the ground state is a Kramers doublet — two states, as required by time-reversal symmetry for an odd number of electrons — its response to a magnetic field is that of a pseudo-spin $\tilde{S} = \frac{1}{2}$,

$$\mathcal{H}_{\text{eff}} = \mu_B \, \vec{B} \cdot \mathbf{g} \cdot \hat{\vec{\tilde{S}}}. \tag{28}$$

Projecting the magnetic moment $M_\alpha = (L + 2S)_\alpha$ on the doublet gives $2 \times 2$ matrices which can always be written as $A_\alpha = \sum_\beta g_{\alpha\beta}\sigma_\beta/2$. The individual $g_{\alpha\beta}$ depend on the arbitrary basis chosen inside the doublet, but the combination

$$G_{\alpha\beta} = \sum_\gamma g_{\alpha\gamma} g_{\beta\gamma} = 2\,\mathrm{Tr}\big[A_\alpha A_\beta\big] \tag{29}$$

does not. TranCI returns the symmetric positive square root $\mathbf{g} = \sqrt{G}$: its eigenvalues are the principal values $g_x, g_y, g_z$ and its eigenvectors the magnetic axes.

```
M = Atom.get_manifolds(H)
g = M.get_gtensor() # 3 x 3 symmetric g-matrix
gs, axes = M.get_principal_g() # principal values and magnetic axes
```

The result is exact for the doublet, not a finite difference. Along an arbitrary direction $\hat{n}$ the predicted splitting is $\Delta E = \sqrt{\hat{n} \cdot \mathbf{g}^2 \cdot \hat{n}}\, \mu_B B$, and this agrees with the directly computed splitting to all six digits of a finite-field check ($b = 10^{-6}$ eV), including for tilted (non-Cartesian) magnetic axes where the off-diagonal elements of $G$ are of the same size as the diagonal ones. A purely spin doublet — a crystal-field split $d^1$ with no spin–orbit coupling — gives $g_x = g_y = g_z = 2$ exactly, while a low-symmetry $d^7$ doublet, e.g. $4\hat{V}_{ee} + 0.6\,\mathtt{z2} + 0.15\,(\mathtt{x2} - \mathtt{y2}) + 0.05\,\mathtt{ls}$, gives strongly anisotropic values (0.81, 1.70, 9.45). A ground state that is not two-fold degenerate raises an error, since a $g$-tensor is not defined for it; the degeneracy is judged with `hamiltonians.tol`, which can be overridden through the `tol` keyword.

## 5.5 Correlation entropy

A single Slater determinant is fully characterized by its one-body density matrix, whose eigenvalues (natural occupations) are then all 0 or 1. Any deviation signals genuine many-body correlation. TranCI quantifies this with the correlation entropy of the one-body density matrix

$$\rho_{ij} = \langle\Psi|\, c_i^\dagger c_j \,|\Psi\rangle, \qquad \rho\,|\phi_k\rangle = n_k\,|\phi_k\rangle, \qquad S = -\sum_k n_k \ln n_k, \tag{30}$$

computed by `get_correlation_entropy(wf)` for any many-body wavefunction `wf` (typically `M.get_gs_manifold()[0]`). $S = 0$ for a single determinant, and $S$ grows as the state becomes a superposition of many configurations, which is exactly what a strong Coulomb interaction produces. Occupations below $10^{-7}$ are discarded to avoid $0 \ln 0$.

### 5.6 Dynamical correlators

Spectroscopies — inelastic tunnelling, neutron scattering, electron spin resonance — measure the frequency-resolved response of the ion to an operator $B$, detected through an operator $A$. TranCI computes

$$S_{AB}(\omega) = -\frac{1}{\pi}\,\mathrm{Im}\,\big\langle \Psi_0 \big| A\,\big[\omega + E_0 + i\delta - \mathcal{H}\big]^{-1} B \big| \Psi_0 \big\rangle \;=\; \sum_n \langle \Psi_0|A|n\rangle\langle n|B|\Psi_0\rangle\,\mathcal{L}_\delta\big(\omega - (E_n - E_0)\big), \tag{31}$$

where $\mathcal{L}_\delta(x) = \frac{1}{\pi}\frac{\delta}{x^2+\delta^2}$ is a normalized Lorentzian of half-width $\delta$. The frequency $\omega$ is measured from the ground state, so peaks appear at the excitation energies returned by `get_excitations()`, with weights given by the matrix elements.

```
import numpy as np
A = Atom.one2many(Atom.SP_Operator["sy"] @ Atom.SP_Operator["dz2"])
es, ds = M.get_dynamical_correlator(A, es=np.linspace(-1., 1., 100))
```

Practical notes:

- `B` defaults to `A`, giving the spectral function of a single operator.
- `delta` (default 0.03 eV) is the Lorentzian broadening. It must be larger than the spacing of the frequency grid `es`, or peaks will fall between grid points.
- The result is *averaged* over the degenerate ground states, which is the right thing for an unpolarized, zero-temperature measurement.
- The default solver (`mode="cv"`) is the correction-vector method: for each frequency a shifted linear system $\big[(\mathcal{H}-\omega)^2+\delta^2\big]x = -\delta B|\Psi_0\rangle$ is solved by conjugate gradients. `mode="full"` inverts the resolvent explicitly and is a useful cross-check. A Chebyshev (kernel polynomial) implementation is also provided in `dynamicstk`.

### 5.7 Effective low-energy Hamiltonians

Deep in the multiplet regime the low-energy physics of the ion is described by a handful of states — a pseudo-spin $S$ with $2S+1$ levels — and by an effective spin Hamiltonian containing anisotropy and Zeeman terms. TranCI extracts that Hamiltonian numerically.

The procedure is:

1. Project the full Hamiltonian on the $n$ lowest eigenstates, $h_{ab} = \langle\psi_a|\mathcal{H}|\psi_b\rangle$, and remove the trace, $h \to h - \frac{\mathrm{Tr}\,h}{n}\mathbb{1}$, since a constant shift carries no physics.
2. Project the chosen operators (by default $S_\alpha$; optionally $L_\alpha$, $J_\alpha$) on the same manifold, and rescale each so that its largest eigenvalue in magnitude equals $S = (n-1)/2$. The projected operators then have the spectrum of genuine spin-$S$ components, and the symbols $\hat{S}_\alpha$ in the output mean what they say.
3. Build an operator basis from those matrices: the identity, the linear terms $\hat{S}_\alpha$, their powers $\hat{S}_\alpha^p$ up to $p = 2$, and symmetrized bilinears $\frac{1}{2}(\hat{S}_\alpha\hat{S}_\beta + \hat{S}_\beta\hat{S}_\alpha)$. Linearly dependent members are discarded.
4. Fit $h$ by least squares,
$$\min_{\{c_a\}} \Big\langle \big| h - \textstyle\sum_a c_a\, O_a \big|^2 \Big\rangle, \tag{32}$$
with real coefficients, using analytic gradients (`jax`) and 40 random restarts to avoid local minima.

5. Emit the result as LaTeX, normalized to the largest coefficient so that the overall energy scale is factored out: $\mathcal{H}_{\text{eff}} = c_{\max}\big[\hat{O}_1 + r_2\hat{O}_2 + \dots\big]$.

The resulting Hermitian model is of the familiar form

$$\mathcal{H}_{\text{eff}} = D\,\hat{S}_z^2 + E\,(\hat{S}_x^2 - \hat{S}_y^2) + \vec{g}\mu_B\vec{B}\cdot\hat{\vec{S}} + \dots, \tag{33}$$

i.e. the axial and rhombic magnetic anisotropies of the ion, derived from first principles rather than assumed.

```
from tranci import effectivehamiltonian
deg = M0.get_gs_multiplicity() # size of the low-energy block
text = effectivehamiltonian.effective_spin_hamiltonian(
           M, H=H, n=deg, operators=["sx","sy","sz"])
print(text) # LaTeX source
```

**Caveats.** The fit only means something if the chosen operator basis can actually represent the block. The code reports the relative error of the fitted spectrum, prints a warning when it exceeds 1%, and inserts a warning in the LaTeX output. If you see one, either enlarge the operator set (add `"lx","ly","lz"`), or reconsider $n$: it should equal the multiplicity of a manifold that is well separated from the rest of the spectrum, not an arbitrary cut through a degenerate group. A second entry point, `effective_hamiltonian(lowest, n, nt)`, tries the $LS$, $SJ$ and $LJ$ operator sets in turn and additionally prints the effective algebra; this is the one used by the graphical interface.

## 6 Custom crystal fields and custom operators

The built-in Cartesian generators of Sec. **??** cover the usual point symmetries, but any one-body operator can be supplied directly. The route is: write a $10 \times 10$ matrix in the spin-orbital basis of Sec. **??**, then promote it to the many-body space with

$$\hat{O} = \sum_{ij} O_{ij}\, c_i^\dagger c_j \tag{34}$$

which is exactly what `Atom.one2many(m)` computes (with the correct fermionic signs, via `edtk.states.one2many_basis`).

To build such matrices, `Atom.SP_Operator` holds the same operator dictionary in $10 \times 10$ single-particle form. It is simply the $n_e = 1$ operator set, so `SP_Operator["lx"]` is the $10 \times 10$ matrix $l_x$, `SP_Operator["dz2"]` the single-particle $d_{z^2}$ projector, and so on.

```
Atom = get_atom(ne=2)
SP = Atom.SP_Operator

# a one-body crystal field written in single-particle language
h1 = 0.1*(SP["x4"] + SP["y4"] + SP["z4"]) + 0.3*SP["z2"] + 0.05*SP["ls"]

H = Atom.one2many(h1) + 4*Atom.Operator["Coulomb"]
M = Atom.get_manifolds(H)
```

This is also the way to compare a one-body (single-particle) description with the correlated many-body result: build the *same* $h_1$ for `ne=1` and for `ne=2`, and compare the orbital occupations. The example `examples/single_occ_VS_many_occ` does precisely that, and shows how the Coulomb interaction redistributes the occupations relative to the non-interacting filling of the same levels.

Any Hermitian $10 \times 10$ matrix is legitimate: a ligand-field matrix obtained from a density-functional calculation, a Wannier-projected on-site Hamiltonian, or a hopping matrix. The only requirement is that the row/column ordering matches Sec. **??**.

# 7 Python API reference

## 7.1 Entry point

| | |
|---|---|
| `get_atom(ne)` | load the operator library for $n_e = 1 \dots 9$ |
| `Atom.Operator` | dict of many-body matrices (dim $\times$ dim) |
| `Atom.SP_Operator` | dict of single-particle matrices ($10 \times 10$) |
| `Atom.one2many(m)` | promote a $10 \times 10$ matrix to many body |
| `Atom.basis` | list of occupation-number basis states |
| `Atom.get_manifolds(H)` | diagonalize, returns `Lowest_States` |
| `Atom.get_latex_wavefunction(v)` | LaTeX expansion of a wavefunction |
| `Atom.wavefunction_z_axis` | quantization axis used when printing states |

## 7.2 Operator keys

| Key | Operator | Notes |
|---|---|---|
| `"sx" "sy" "sz"` | $S_\alpha = \sum_i (s_\alpha)_i$ | total spin |
| `"lx" "ly" "lz"` | $L_\alpha = \sum_i (l_\alpha)_i$ | total orbital momentum |
| `"jx" "jy" "jz"` | $J_\alpha = L_\alpha + S_\alpha$ | total angular momentum |
| `"s2" "l2" "j2"` | $S^2$, $L^2$, $J^2$ | true many-body squares |
| `"Coulomb", "vc"` | $\hat{V}_{ee}$ | Slater–Condon, see Sec. **??** |
| `"ls"` | $\sum_i \vec{l}_i \cdot \vec{s}_i$ | spin–orbit coupling |
| `"x2" "y2" "z2"` | $\sum_i (l_\alpha^2)_i$ | *not* $L_\alpha^2$ |
| `"x4" "y4" "z4"` | $\sum_i (l_\alpha^4)_i$ | cubic invariant if summed |
| `"x2y2"` | $\sum_i [(l_x l_y)^2 + (l_y l_x)^2]_i$ | |
| `"dz2" "dx2y2"` | $\hat{n}_\mu$ | $e_g$ occupations |
| `"dxy" "dxz" "dyz"` | $\hat{n}_\mu$ | $t_{2g}$ occupations |
| `"m0" "m1" "m2"` | projectors on $\lvert m\rvert = 0, 1, 2$ | |

## 7.3 `Lowest_States`

| | |
|---|---|
| `.evals`, `.evecs`, `.e0`, `.h`, `.atom` | spectrum and inputs |
| `get_gs_degeneracy(tol=None)` | tuple $(n_{\mathrm{GS}}, E_{\mathrm{GS}})$ |
| `get_gs_multiplicity()` | the integer degeneracy |
| `get_degeneracies()` | $[(n_k, E_k)]$ for all manifolds |
| `get_excitations()` | $[E_k - E_0]$ in eV |
| `get_manifolds()` | eigenvectors grouped by manifold |
| `get_gs_manifold()` | ground-state eigenvectors |
| `get_representation(A,n=6)` | $\langle \psi_a \lvert A \rvert \psi_b \rangle$ |
| `get_gs_projected_eigenvalues(A)` | eigenvalues of $A$ in the GS manifold |
| `disentangle_manifolds(A)` | re-label all manifolds by $A$ |
| `disentangle_gs_manifold(A)` | re-label the GS manifold by $A$ |
| `get_correlation_entropy(wf)` | $-\sum_k n_k \ln n_k$ |
| `get_gtensor(tol=None)` | $3 \times 3$ $g$-matrix of a Kramers doublet |
| `get_principal_g(tol=None)` | principal $g$ values and magnetic axes |
| `get_dynamical_correlator(A,B=None,...)` | $(\omega, S_{AB}(\omega))$ |

## 7.4 Worked examples shipped with the code

Each directory under `examples/` contains a self-contained `main.py`, to be run from inside that directory.

`octahedral` Builds a cubic field directly from powers of the total orbital operators, $L_x^4+L_y^4+L_z^4$, adds a small uniaxial term and the Coulomb interaction for $n_e = 1$, and prints the ground-state degeneracy together with the occupation of each cubic orbital. The fastest sanity check of an installation (seconds).

`orbital_projection` Sweeps a uniaxial crystal field for $n_e = 3$ and plots, as a function of its strength, the five orbital occupations, the $S_z$ content of the ground-state manifold, and the ground-state degeneracy. The total spin stays $S = 3/2$ across the sweep while the ground-state degeneracy drops from 8 to 4: the crystal field quenches the orbital degree of freedom, not the spin.

`single_occ_VS_many_occ` Compares the orbital occupations of a correlated $d^2$ ion with those of a single electron subject to the same one-body field, illustrating the effect of the Coulomb interaction. Also the reference for building custom crystal fields through `one2many`.

`correlation_entropy` Computes the correlation entropy of the ground state of a $d^3$ ion with crystal field, Zeeman and spin–orbit terms.

`dynamical_spin_correlator` Computes the spin excitation spectrum of a $d^6$ ion, using an operator that combines a spin flip with an orbital projector, and plots $S_{AA}(\omega)$.

`effective_hamiltonian` Fits the low-energy block of a $d^2$ ion with spin–orbit coupling to an effective spin model and prints the resulting LaTeX formula. Exercises the `jax`-based fitting path.

The notebooks in `notebooks/` cover the same ground interactively: `Hund_rules.ipynb`, `orbital_CF.ipynb`, `anisotropy_VS_crystal_field.ipynb`, `custom_crystal_field.ipynb`, `Zeeman_splitting.ipynb` and `Effective_Hamiltonian.ipynb`.

# 8 The graphical interface

The graphical front-end is started with `tranci` on Linux and Mac (`bin\tranci.bat` on Windows) after running `python install.py`, which only adds `bin/` to the `PATH`. It needs `PyQt5` and a `pdflatex` installation (MiKTeX on Windows, TeX Live or MacTeX elsewhere).

## 8.1 Parameters

All parameters are visible at once in the `Parameters` panel, grouped as `Atom`, `Crystal field` and `Zeeman and exchange fields`. The interface builds a fixed Hamiltonian from the fields below. Each field multiplies exactly one operator of Sec. **??**; all of them are in eV except `U`, which is a dimensionless multiplier (Sec. **??**). Hovering over any field shows a tooltip with its meaning. The number of electrons and the other integer fields are spin boxes and only accept values in their valid range; the real-valued fields reject anything that is not a number as it is typed.

Next to every field the interface shows the operator it multiplies, written in second quantization and rendered from TeX at start-up (hovering over a formula shows its TeX source). Here $c^\dagger_{m\sigma}$ creates an electron with $l_z = m$ ($m = -2, \dots, 2$) and spin $\sigma$ in the $d$ shell, lower-case $l, s$ are single-particle operators and $V_{ijkl}$ is the tensor of Eq. (**??**):

| Field | Second-quantized operator |
|---|---|
| `# of electrons` | $\hat{N} = \sum_{m\sigma} c^\dagger_{m\sigma} c_{m\sigma}$ |
| `U` | $U \sum_{ijkl} \sum_{\sigma\sigma'} V_{ijkl}\, c^\dagger_{i\sigma} c^\dagger_{j\sigma'} c_{k\sigma'} c_{l\sigma}$ |
| `SOC` | $\lambda \sum_{mm'\sigma\sigma'} \langle m\sigma \vert \vec{l}\cdot\vec{s} \vert m'\sigma'\rangle\, c^\dagger_{m\sigma} c_{m'\sigma'}$ |
| `Uniaxial z^2` | $D \sum_{m\sigma} m^2\, c^\dagger_{m\sigma} c_{m\sigma}$ |
| `Shear x^2-y^2` | $E \sum_{mm'\sigma} \langle m \vert l_x^2 - l_y^2 \vert m'\rangle\, c^\dagger_{m\sigma} c_{m'\sigma}$ |
| `Octahedral` | $O \sum_{mm'\sigma} \langle m \vert l_x^4 + l_y^4 + l_z^4 \vert m'\rangle\, c^\dagger_{m\sigma} c_{m'\sigma}$ |
| `Trigonal` | $t \sum_{mm'\sigma} \langle m \vert (\vec{l}\cdot\hat{n})^2 \vert m'\rangle\, c^\dagger_{m\sigma} c_{m'\sigma}, \quad \hat{n} = (1,1,1)/\sqrt{3}$ |
| `Uniaxial' z^4` | $D' \sum_{m\sigma} m^4\, c^\dagger_{m\sigma} c_{m\sigma}$ |
| `Shear' (xy)^2+(yx)^2` | $E' \sum_{mm'\sigma} \langle m \vert (l_x l_y)^2 + (l_y l_x)^2 \vert m'\rangle\, c^\dagger_{m\sigma} c_{m'\sigma}$ |
| `B` | $\vec{B} \cdot \sum_{mm'\sigma\sigma'} \langle m\sigma \vert \vec{l} + 2\vec{s} \vert m'\sigma'\rangle\, c^\dagger_{m\sigma} c_{m'\sigma'}$ |
| `J` | $\vec{J} \cdot \sum_{m\sigma\sigma'} \vec{s}_{\sigma\sigma'}\, c^\dagger_{m\sigma} c_{m\sigma'}, \quad \vec{s} = \vec{\sigma}/2$ |
| `Theta`, `Phi` | $\hat{B} = (\sin\pi\theta_B \cos\pi\phi_B,\ \sin\pi\theta_B \sin\pi\phi_B,\ \cos\pi\theta_B)$, and likewise $\hat{J}$ |

The uniaxial fields are diagonal in $m$ because $l_z\vert m\rangle = m\vert m\rangle$; the matrix elements of the other one-body operators are the $10 \times 10$ matrices of `Atom.SP_Operator` (Sec. **??**).

| Field | Operator | Meaning |
|---|---|---|
| `# of electrons` | — | $n_e$, from 1 to 9 |
| `U [multiplier]` | `vc` | multiplier of the Coulomb tensor (dimensionless) |
| `SOC [eV]` | `ls` | $\lambda$, spin–orbit coupling |
| `Uniaxial z^2` | `z2` | $D$, tetragonal field |
| `Shear x^2-y^2` | `x2-y2` | $E$, rhombic field |
| `Octahedral x^4+y^4+z^4` | `x4+y4+z4` | $O$, cubic field ($10Dq = 6O$) |
| `Uniaxial' z^4` | `z4` | higher-order axial field |
| `Shear' (xy)^2+(yx)^2` | `x2y2` | higher-order in-plane field |
| `Trigonal (x+y+z)^2` | rotated `z2` | field along $(1,1,1)$ |
| `B [eV]`, `Theta_B`, `Phi_B` | $\vec{b}\cdot(\vec{L} + 2\vec{S})$ | Zeeman field |
| `J [eV]`, `Theta_J`, `Phi_J` | $\vec{J}\cdot\vec{S}$ | exchange field (spin only) |

The angles $\theta$ and $\phi$ are given *in units of* $\pi$, so $\theta = 0.5,\ \phi = 0$ means a field along $x$. The direction is $\vec{b} = b(\sin\pi\theta\cos\pi\phi,\ \sin\pi\theta\sin\pi\phi,\ \cos\pi\theta)$.

The `Calculation` panel has three tabs. `Run` holds the single-shot calculation; its checkbox `Fit effective Hamiltonian with` $n$ `states` adds the fit of Sec. **??** to the summary (off by default, since it requires `jax`), with $n$ the dimension of the local effective model, for example $2S+1$. `Sweep` holds the parameter sweeps, with `Levels to plot` limiting how many eigenvalues are drawn (0 draws all). Under `Settings`, `Energy tolerance [eV]` sets the degeneracy-grouping window `tol` discussed above, and the `z axis for the wavefunctions` rotates the quantization axis used when the wavefunctions are printed, so that a state polarized along an arbitrary direction collapses onto a single basis ket.

The status bar at the bottom of the window reports what the interface is doing (the progress of a sweep, whether the PDF was created, where the data were saved); errors open a dialog. The window is blocked while a calculation runs.

## 8.2 Saving and loading parameters

`File` → `Save parameters` (Ctrl+S) writes every field of the interface to a JSON file, and `File` → `Load parameters` (Ctrl+O) fills the interface back from one, so that a calculation can be

reproduced or shared. Entries of the file that do not match a field of the current version are reported and ignored. Every `Initialize and run` also writes the parameters it used to `parameters.json` next to the results, and `Save data` copies that file along with them. `Help` → `Manual` (F1) opens this document.

### 8.3 Tasks

**Initialize and run** performs one calculation and writes `spectrum_ci.tex`, compiling it twice with `pdflatex` (so that the table of contents is correct) into `spectrum_ci.pdf`. The summary contains the Hamiltonian, the table of manifolds with degeneracies and excitation energies, a table of energies with $\langle L_\alpha \rangle$ and $\langle S_\alpha \rangle$ for each state, the projection of all standard operators on the ground-state manifold, the explicit wavefunctions of every manifold, and (optionally) the effective Hamiltonian.

**Show pdf** opens that summary with the system viewer.

**Plot spectrum / Plot excitations** sweep the parameter selected in `Parameter to sweep` over `[Initial, Final]` in `Steps` points and plot all eigenvalues, referred to their centre of gravity or to the ground state respectively. The entries of the sweep list carry the same names as the fields of the `Parameters` panel; the value typed in the swept field is ignored, all the others are kept fixed. The angles are swept in units of $\pi$.

**Plot degeneracy** plots the ground-state degeneracy along the sweep — the quickest way to locate level crossings and spin-state transitions.

**Plot operator** plots the eigenvalues of the operator selected next to the button, projected on the ground-state manifold along the sweep.

**Save data** copies the generated `*.tex`, `*.pdf`, `*.OUT` and `parameters.json` files into `./tranci_data` relative to the directory from which `tranci` was launched, and reports the folder. It is also available as `File` → `Save data` (Ctrl+D).

The interface runs inside a fresh temporary directory, so all output files are created there until `Save data` is pressed.

### 8.4 Output files

| | |
|---|---|
| `spectrum_ci.tex` / `.pdf` | full LaTeX summary of one calculation |
| `parameters.json` | every field of the interface for that calculation |
| `SPECTRUM.OUT` | energy (GHz, meV) and $\langle L_\alpha \rangle$, $\langle S_\alpha \rangle$ |
| `POPULATION.OUT` | energy and the ten spin-orbital occupations |
| `EIGENVALUES.OUT` | swept parameter vs. eigenvalues |
| `DEGENERACY.OUT` | swept parameter vs. ground-state degeneracy |
| `OPERATOR_VALUES.OUT` | swept parameter vs. projected operator eigenvalues |

## 9 Validation

### 9.1 Hund's first and second rules

A direct test of the Coulomb tensor and of the many-body machinery is that the interaction alone, with no crystal field and no spin–orbit coupling, must reproduce the free-ion ground terms predicted by the first two Hund rules: maximize $S$, then maximize $L$. Diagonalizing $\hat{V}_{ee}$ by itself and reading off $\langle S^2 \rangle$ and $\langle L^2 \rangle$ in the ground-state manifold gives

| $n_e$ | Hilbert dim. | $S$ | $L$ | term | degeneracy | $(2S{+}1)(2L{+}1)$ |
|---|---|---|---|---|---|---|
| 1 | 10 | 1/2 | 2 | $^2D$ | 10 | 10 |
| 2 | 45 | 1 | 3 | $^3F$ | 21 | 21 |
| 3 | 120 | 3/2 | 3 | $^4F$ | 28 | 28 |
| 4 | 210 | 2 | 2 | $^5D$ | 25 | 25 |
| 5 | 252 | 5/2 | 0 | $^6S$ | 6 | 6 |
| 6 | 210 | 2 | 2 | $^5D$ | 25 | 25 |
| 7 | 120 | 3/2 | 3 | $^4F$ | 28 | 28 |
| 8 | 45 | 1 | 3 | $^3F$ | 21 | 21 |
| 9 | 10 | 1/2 | 2 | $^2D$ | 10 | 10 |

Every entry matches the textbook free-ion term, including the $L = 0$ half-filled $^6S$ case and the particle–hole symmetry between $n_e$ and $10 - n_e$. The degeneracies come out exactly as $(2S + 1)(2L + 1)$, confirming that the ground manifold is a single $LS$ term. This calculation is reproduced by

```
from tranci import hamiltonians
hamiltonians.tol = 1e-6
Atom = get_atom(ne=2)
M = Atom.get_manifolds(Atom.Operator["Coulomb"])
gs = M.get_gs_manifold()
s2 = hamiltonians.get_projected_eigenvalues(gs, Atom.Operator["s2"]).mean()
l2 = hamiltonians.get_projected_eigenvalues(gs, Atom.Operator["l2"]).mean()
```

### 9.2 The $d^2$ multiplet spectrum

A sharper test uses the full excited-term structure. For $n_e = 2$ the Coulomb interaction alone must split the 45 states into the five free-ion terms $^3F$, $^1D$, $^3P$, $^1G$, $^1S$, at energies given exactly by the Racah parameters. With the multiplier set to $u = 1$, `get_excitations()` returns

| Term | Degeneracy | Racah expression | $\Delta E$ [eV] |
|---|---|---|---|
| $^3F$ | 21 | 0 | 0 |
| $^1D$ | 5 | $5B + 2C$ | 0.68488 |
| $^3P$ | 9 | $15B$ | 0.79301 |
| $^1G$ | 9 | $12B + 2C$ | 1.05495 |
| $^1S$ | 1 | $22B + 7C$ | 2.63497 |

The four excitation energies are reproduced to every printed digit by the single pair $B = 52.87$ meV, $C = 210.3$ meV quoted in Sec. **??**, and the degeneracies $21, 5, 9, 9, 1$ sum to 45. This simultaneously validates the Coulomb tensor, the fermionic signs of the many-body construction and the degeneracy grouping.

### 9.3 Hund's third rule

Hund's *third* rule is not contained in $\hat{V}_{ee}$; it follows from the spin–orbit term. Adding $\lambda\,$`ls` with $\lambda > 0$ — the physical one-electron constant, positive for every filling — yields $J = |L - S|$ below half filling and $J = L + S$ above it, as tabulated in Sec. **??**. No sign change of the input is needed.

### 9.4 Algebraic consistency

Additional internal consistency checks are performed by `tranci.check.check_all(atom)`, which verifies the angular-momentum algebra and the commutation of $\vec{L}$ and $\vec{S}$ with the Coulomb operator.

## 10 Practical notes and limitations

- **Only the $d$ shell.** The model contains no ligand orbitals and no charge fluctuations: the electron number $n_e$ is fixed. Charge-transfer physics, ligand-to-metal hybridization and covalency are represented only indirectly, through the effective crystal-field parameters and through a reduced (nephelauxetic) Coulomb multiplier.
- **The Coulomb prefactor is dimensionless.** See Sec. **??**. Only the ratios $B$ and $C$ affect the multiplet splittings at fixed $n_e$.
- **Crystal-field keys are sums of one-body operators.** `x2` $\neq L_x^2$. Use `"l2"`, `"s2"`, `"j2"` when you want the many-body squares.
- **Degeneracy is a numerical statement.** It depends on `hamiltonians.tol`. Always set it deliberately when a small splitting matters.
- **Dense diagonalization.** Every call to `get_manifolds` builds a dense matrix and computes the full spectrum, at $O(\dim^3)$ cost. This is negligible for a single point, but a sweep with many steps repeats it at every step; reuse the `Atom` object (loading it re-reads files from disk) and keep sweeps modest.
- **Silent zeros on malformed files.** `read.read_matrix` wraps its parsing in a bare `except` and returns a zero matrix if anything goes wrong, rather than raising. A corrupted or truncated `.op` file therefore shows up as a missing term in the Hamiltonian, not as an error. If a spectrum looks unexpectedly degenerate, verify the operator library.
- **The $g$-tensor needs a Kramers doublet.** `get_gtensor` is defined only for a two-fold degenerate ground state and raises otherwise (Sec. **??**). For a larger manifold, read the anisotropy off the effective Hamiltonian of Sec. **??** instead.
- **Effective-Hamiltonian fits need checking.** The fit is a numerical least-squares problem in a chosen operator basis; always look at the reported relative error before quoting the coefficients.
- **Hyperfine coupling is library-only.** It is not reachable from the graphical interface, and only spin-$\frac{1}{2}$ nuclei are implemented.
- **Dead code.** `src/tranci.py` (the old GTK front-end), `src/tranci/states.py` and `src/tranci/fix_coulomb.py` are legacy and are not imported by anything. `interface_pyqt/interface.py` is generated from `interface.ui` and must not be edited by hand.